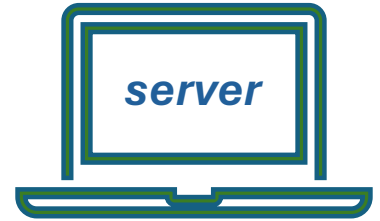


Shiny: Understanding the server

Ermis-Ioannis Michail-Delopoulos

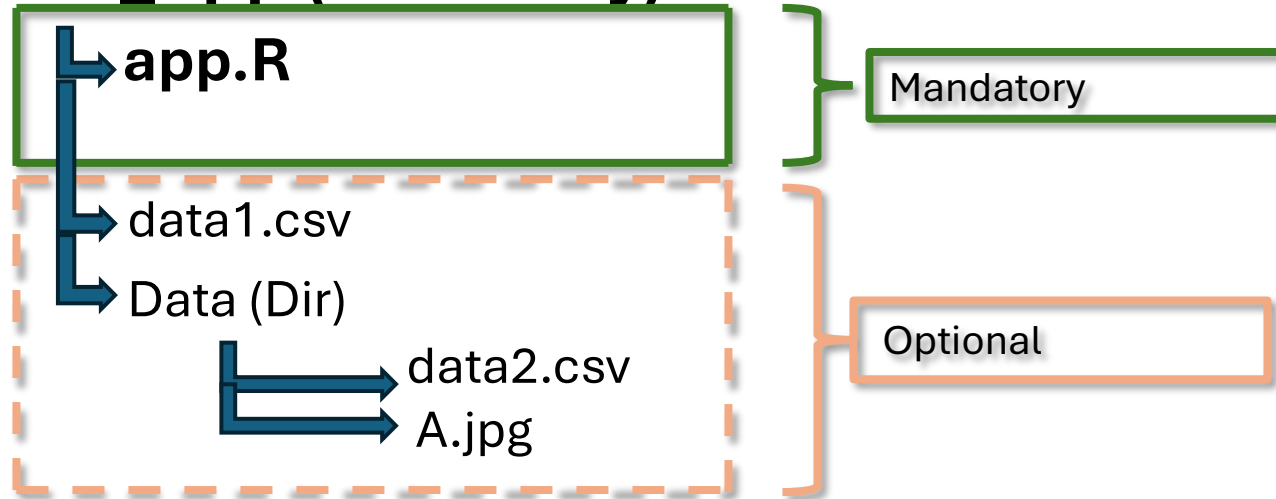
Learning Objectives



- **Understand the role of the server function in Shiny**
- **Learn how to link UI components (inputs & outputs) with server-side logic**
- **Grasp the concepts of reactivity and reactive context**

Quick Recap: Structure of Shiny App

- **Test_App (Directory)**



The name of the app is assigned at the directory

```
shiny::runApp("Test_App")
```

Quick Recap: App.R

- Comprises of 3 parts
 - User Interface (Ui.R)
 - Server function (server.R)
 - Shiny app function

```
# Import Packages
library(shiny)           # Imported Packages/libraries we need for shiny app
                          # At least 1 package loaded(shiny)

# Global variables loaded here

ui <- fluidPage(
  # Code for Ui logic
)

server <- function(input, output, session) {
  # Code for Server logic
}

shinyApp(ui = ui, server = server)
```

What happens when we start a shiny app

1. Initialization (**Once**)

- Objects, packages, and data defined outside the server() function are loaded
- UI and the HTML blueprint are compiled

2. Server Setup (**Once** per user session)

- Shiny creates a new environment and calls the server() function
- Each user session runs in its own isolated environment

3. Reactive Lifecycle (**Continuously** during the session)

- Tracks input-output dependencies
- Re-runs render*() blocks and updates outputs dynamically when inputs change

The server function

Server() function has 3 parameters:

1. *input*
2. *output*
3. *session*

```
server <- function(input, output, session){...}
```

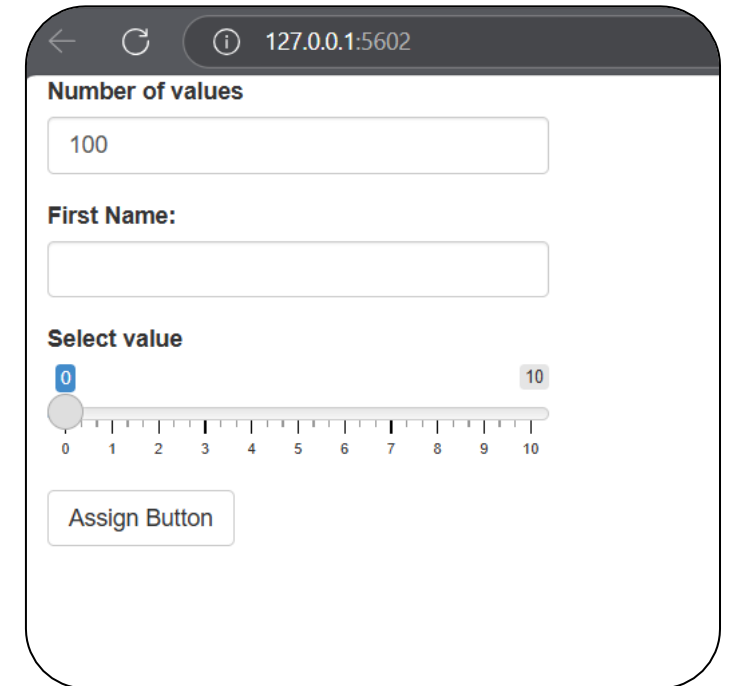
Handles advanced tasks (updates, URLs...)

*List-like, reactive, **READ-ONLY***

*List-like, reactive, **READ-ONLY***

Inputs

```
ui <- fluidPage(  
  numericInput("count", label = "Number of values", value = 100),  
  textInput(inputId = "txt1", label="First Name:"),  
  sliderInput(inputId = "slider1", min=0, max=10, label="Select value",value=0),  
  actionButton(inputId="assign", "Assign Button"),  
  
  textOutput(outputId = "textOutput1")  
)
```



Inside the server, we can access any input components value with the accessor `input$` and the **inputId**:

input

\$ count	→ numeric (e.g., 42)
\$ txt1	→ character (e.g., "Alice")
\$ slider1	→ numeric (e.g., 3)

BUT, unlike a typical list, input objects are:

- **Read-only** (cannot be directly manipulated)
- To handle them you need **reactive context** (more later on)

```
server <- function(input, output, session) {  
  input$count <- 10  
}  
  
shinyApp(ui, server)  
#> Error: Can't modify read-only reactive value 'count'
```

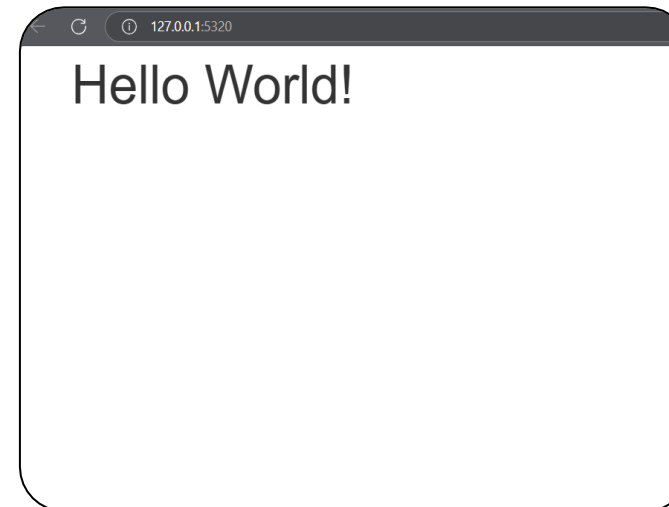


Outputs

Outputs are similar to Inputs:

- List-like objects named according to the **outputId**, but instead of receiving, they sent output to the UI
- **Outputs** cannot be directly handled, they need a **reactive context**, which in this case is **render*() functions**.
- **Outputs** are **always** accompanied with **render*() functions**.

```
ui <- fluidPage(  
  
  # Output control/widget in Ui  
  textOutput(outputId = "textOutput1")  
  
)  
  
server <- function(input, output, session) {  
  
  # Create the output to send to Ui to visualize  
  output$textOutput1 <- renderText("Hello World!")  
  
}  
  
shinyApp(ui = ui, server = server)
```



Reactive context

- To access **inputs** and **outputs** we need to wrap them inside of **reactive context**

1. Reactive expressions

- `reactive({ ... })`
 - `eventReactive(event,{ ... })`
- 

2. Rendering functions

- `renderPlot ({ ... })`
- `renderTable ({ ... })`
- `renderText ({ ... })`,
- `etc.`

3. Observers

- `observe({ ... })`
- `observeEvent(event, { ... })`

- Can access reactive inputs or other reactive expressions
- Automatically track a user input
- Cache the results & Return a value

Reactive

```
ui<-fluidPage(  
  numericInput(inputId = "count",label = "Give number", value=10)  
)
```

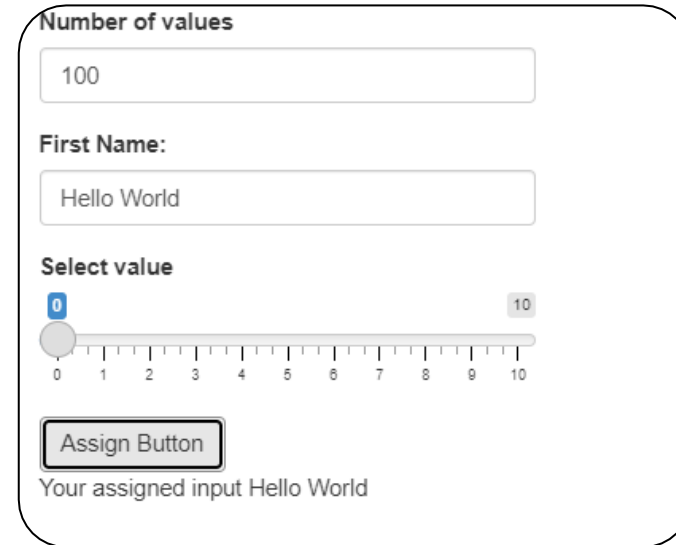


```
server<-function(input,output,session){  
  x<-reactive({ input$count })  
  y<-reactive({  
    tmp<- x()+1  
    tmp/2  
  })  
}
```

After creating a reactive
variable to call it again
we need parenthesis,
**So, to call x we call it
as x()**

EventReactive

```
server <- function(input, output, session) {  
  
  input_value <- eventReactive(input$assign,{input$txt1 })  
  
  output$textOutput1<-renderText({  
    | paste("Your assigned input", input_value())  
    | })  
}
```



Number of values

100

First Name:

Hello World

Select value

0 10

0 1 2 3 4 5 6 7 8 9 10

Assign Button

Your assigned input Hello World

Similarly to previous reactive example, we can call **eventReactive** in the textOutput1 **input_value()**

Reactive context

- To access **inputs** and **outputs** we need to wrap them inside of **reactive context**

1. Reactive expressions

- `reactive({ ... })`
- `eventReactive(event,{ ... })`

2. Rendering functions

- `renderPlot ({ ... })`
- `renderTable ({ ... })`
- `renderText ({ ... })`,
- `etc.`

3. Observers

- `observe({ ... })`
- `observeEvent(event, { ... })`

- 
- Automatically tracks what inputs the output uses
 - Converts the output of your R code into HTML suitable for display on a web page

Render functions

Types of output (in Ui)

Output function	Creates
<code>dataTableOutput</code>	DataTable
<code>htmlOutput</code>	raw HTML
<code>imageOutput</code>	image
<code>plotOutput</code>	plot
<code>tableOutput</code>	table
<code>textOutput</code>	text
<code>uiOutput</code>	raw HTML
<code>verbatimTextOutput</code>	text

Render* functions in server that send the reactive objects back to Ui

render function	creates
<code>renderDataTable</code>	DataTable
<code>renderImage</code>	images (saved as a link to a source file)
<code>renderPlot</code>	plots
<code>renderPrint</code>	any printed output
<code>renderTable</code>	data frame, matrix, other table like structures
<code>renderText</code>	character strings
<code>renderUI</code>	a Shiny tag object or HTML

Example Render function

```
ui <- fluidPage(  
  imageOutput("image")  
)
```

```
server <- function(input, output, session) {  
  output$image<-renderImage({  
    list(src = "data/data-science-500x500.jpg", height = "100%")  
  },  
  deleteFile = FALSE  
)  
}
```



Reactive context

- To access **inputs** and **outputs** we need to wrap them inside of **reactive context**

1. Reactive expressions

- `reactive({ ... })`
- `eventReactive(event,{ ... })`

2. Rendering functions

- `renderPlot ({ ... })`
- `renderTable ({ ... })`
- `renderText ({ ... })`,
- `etc.`

3. Observers

- `observe({ ... })`
- `observeEvent(event, { ... })`

- can access reactive inputs and reactive expressions, but they don't return a value
- they are used for their side effects

Observers

`observe({ ... })`

```
server <- function(input, output, session) {  
  
  input_value <- reactive({input$txt1})  
  
  observe({  
    showNotification(input_value(), type = "message")  
  })  
}
```

`observeEvent(event, { ... })`

```
server <- function(input, output, session) {  
  
  input_value <- eventReactive(input$assign,{input$txt1 })  
  
  observeEvent(input$assign, {  
    showNotification("Button clicked!", type = "message")  
  })  
  
  output$textOutput1<-renderText({  
    paste("Your assigned input", input_value())  
  })  
}
```

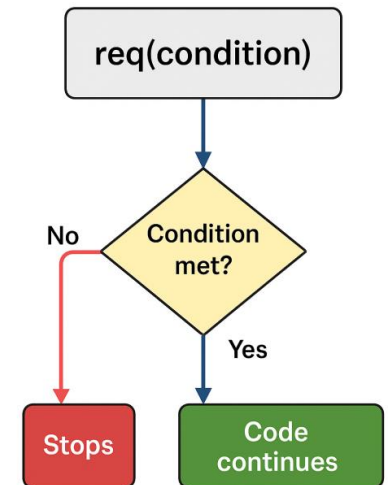
Using req() in Shiny

💡 What is req()?

- A validation function that ensures required conditions or inputs are available before proceeding
- If the condition is not met, Shiny stops the reactive or render function where req() is called

✅ When to use it?

- To prevent errors when inputs are empty or NULL
- To stop downstream code until a user selects or provides input



Using req() in Shiny

```
server <- function(input, output, session) {  
  
  input_value1 <- reactive({input$txt1 })  
  input_value2 <- reactive({input$txt2})  
  
  output$textOutput1 <- renderText({  
    paste("Your Name:", req(input_value1()), "Your Last Name:", req(input_value2()))  
  })  
}
```

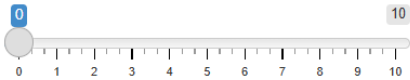
Number of values

First Name:

Last Name:

Select value

0 10



0 1 2 3 4 5 6 7 8 9 10



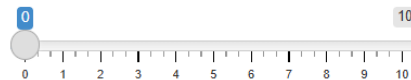
Number of values

First Name:

Last Name:

Select value

0 10



0 1 2 3 4 5 6 7 8 9 10

Your Name Hello Your Last Name World

Notifications

- In Shiny, notifications are created with **showNotification()**, and stack in the bottom right of the page.

```
showNotification(  
  ui,  
  action = NULL,  
  duration = 5,  
  closeButton = TRUE,  
  id = NULL,  
  type = c("default", "message", "warning", "error"),  
  session = getDefaultReactiveDomain()  
)
```

Hands-On: Complete the Shiny Server

Grab the starter code

- Clone / download from Github: [code_exercises_L3_Exercise](#).

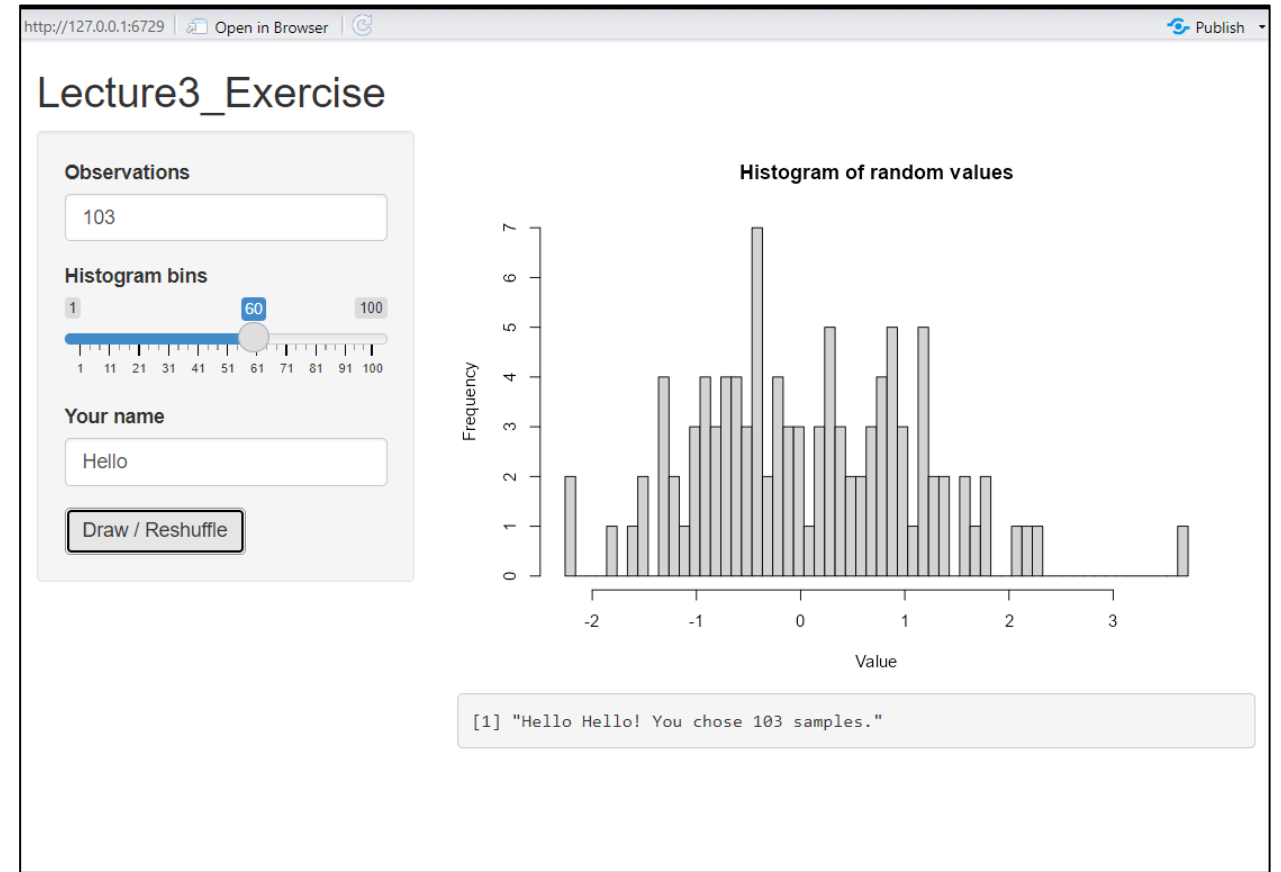
Open App.R

- The UI is finished: your mission is to **fill in** the numbered **TODOs** in the **server block**.

Build & test

- Implement TODO, click Run App, confirm It works

Approximate time: 15–20 minutes.



How app should look in the end

The reactive graph

Reactive graph helps us to understand the order of execution, which describes how inputs and outputs are connected

Three components of reactive execution in Shiny are:

1. reactive inputs



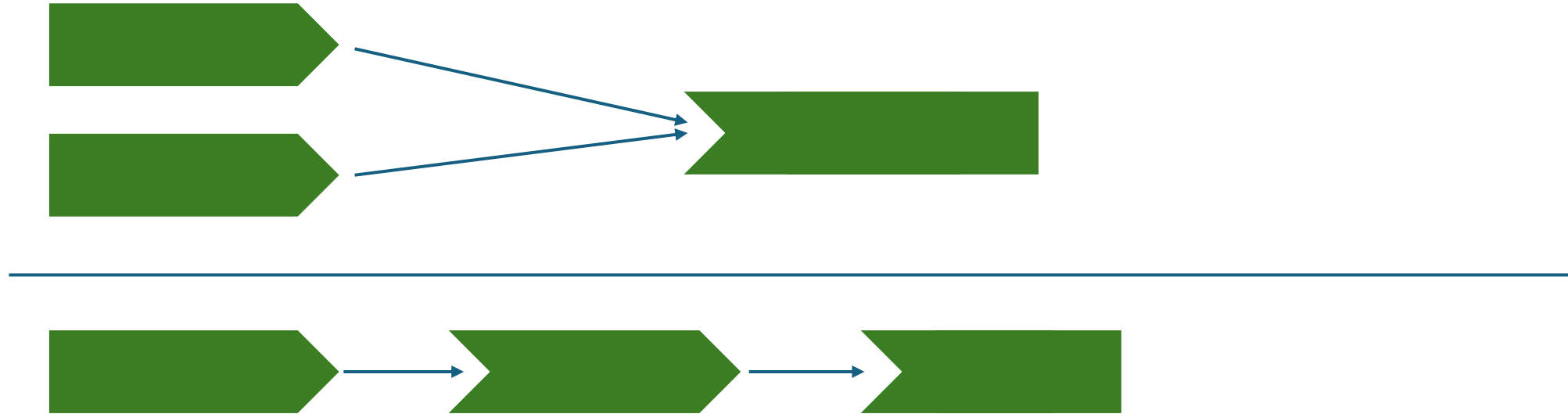
2. reactive expressions



3. reactive outputs

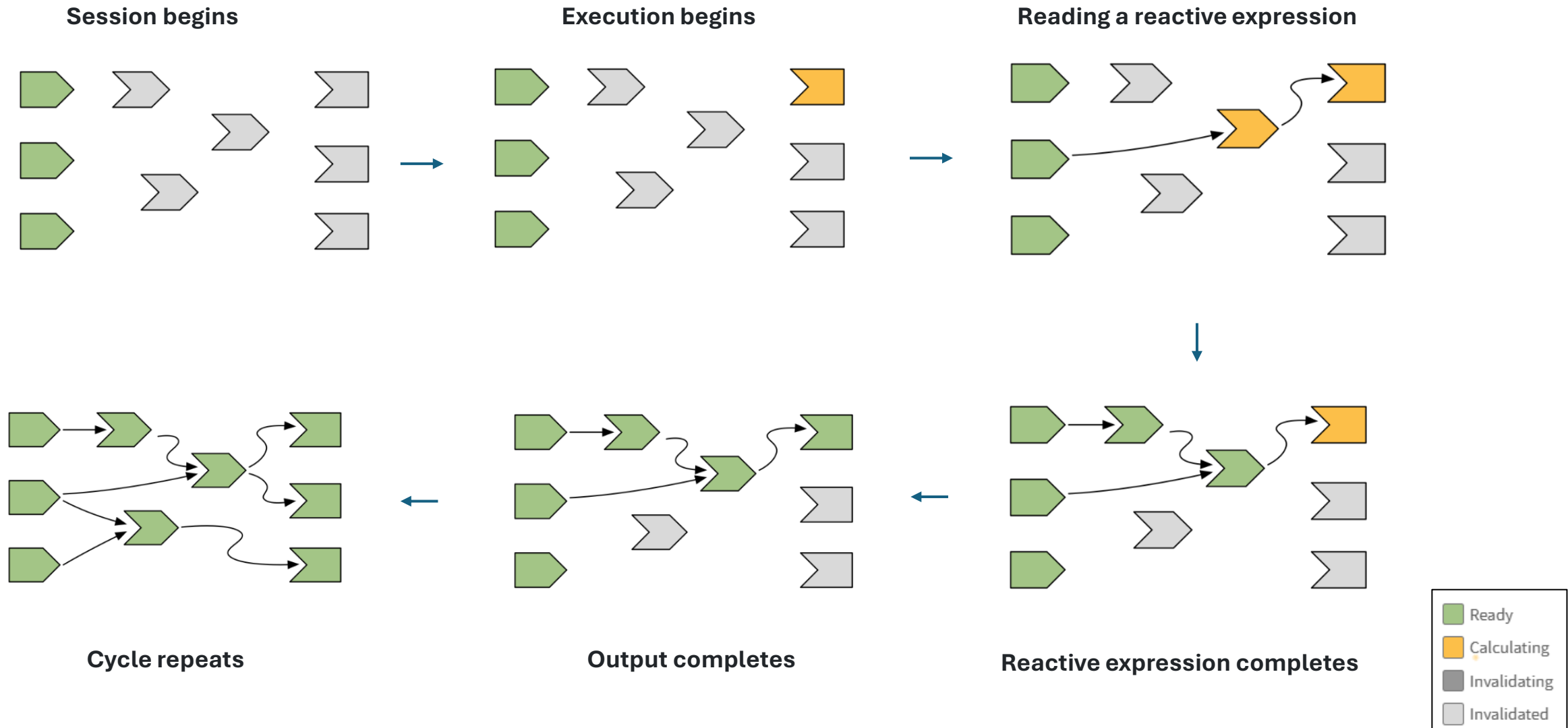


The reactive graph

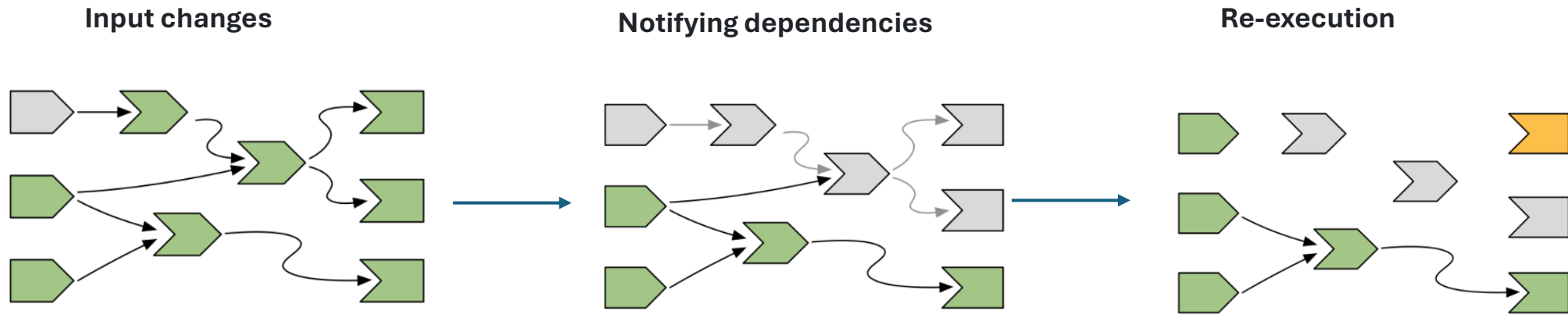


The connections between the components are directional, with the arrows indicating the direction of reactivity

The reactive graph



The reactive graph



The reactive graph - reactlog package

Using **reactlog** package

```
# Install reactlog package :
```

```
>> install.packages("reactlog")
```

```
# Add reactlog library
```

```
>> library(shiny)
```

```
>> library(reactlog)
```

```
# tell shiny to log all reactivity
```

```
>> reactlog_enable()
```

```
# run a shiny app
```

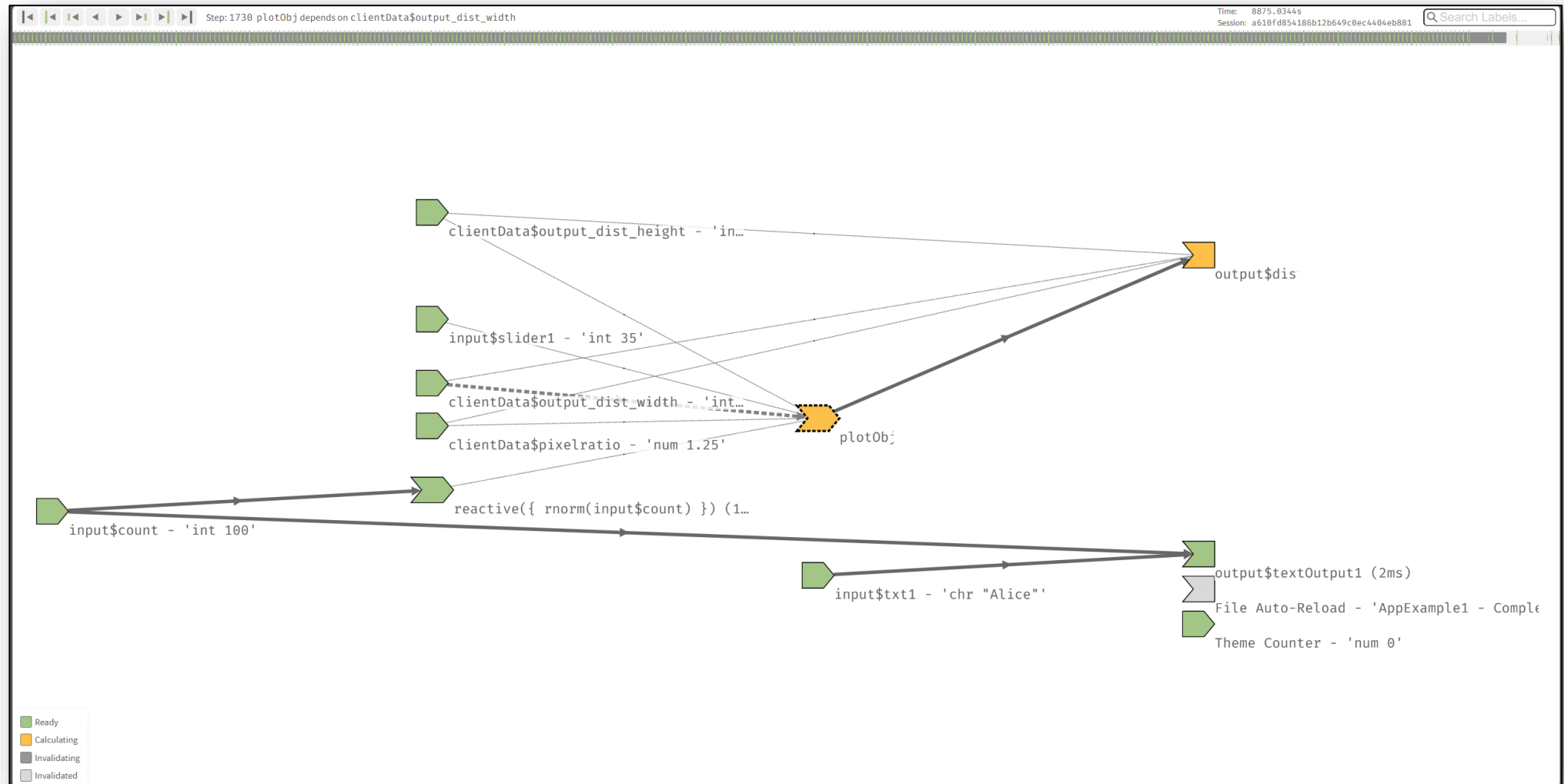
```
>> shiny::runApp(app) #app (directory where app.R is)
```

```
# once app has closed, display reactlog from shiny
```

```
>>shiny::reactlogShow()
```

Or while Shiny app is running, press Ctrl+F3 (Mac: Cmd+F3) to launch the reactlog application

The reactive graph – reactlog package



Reactive programming – Shiny Reactivity

💡 The idea is that you don't need to tell shiny when to update the inputs and outputs. But Shiny does it automatically

- In **imperative** programming , you issue a specific command, and it's carried out immediately, like in R.
- In **declarative/reactive** programming, you express higher-level goals or describe important constraints. This is the style of programming Shiny uses

